Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 4



Nama:

Muhammad Yusuf Bagus Syafaat

NIM:
20240801041

UNIVERSITASA ESA UNGGUL TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

2026

1. Pendahuluan

1.1 Latar Belakang

Aplikasi mobile sering membutuhkan data yang berasal dari server, misalnya data pengguna, artikel, atau komentar. Pada Flutter, data tersebut dapat diambil melalui REST API menggunakan permintaan HTTP. Karena proses komunikasi melalui internet membutuhkan waktu, aplikasi perlu menangani proses asinkron agar tampilan tetap responsif.

1.2 Tujuan Praktikum

- Memahami konsep Future, async, dan await pada Dart.
- Mengambil data dari REST API menggunakan paket http dan menguraikan respons JSON.
- Membuat class model dengan konstruktor fromJson.
- Menampilkan data asinkron menggunakan FutureBuilder dengan kondisi loading, error, dan data.
- Menangani kegagalan jaringan serta menyediakan fitur untuk mencoba kembali.

1.3 Ruang Lingkup

Praktikum mencakup simulasi Future, konfigurasi paket HTTP, pembuatan model Pengguna, pengambilan data dari endpoint JSONPlaceholder, penampilan daftar dan detail pengguna, serta pengujian kondisi galat. Tugas akhir pada modul adalah membuat aplikasi Daftar Postingan dengan daftar komentar.

2. Dasar Teori

2.1 REST API dan HTTP

REST API memungkinkan aplikasi meminta data dari layanan melalui URL tertentu. Pada praktikum ini, permintaan GET digunakan untuk mengambil data dari JSONPlaceholder. Respons server memiliki kode status; kode 200 menunjukkan permintaan berhasil, sedangkan kode seperti 404 menunjukkan sumber tidak ditemukan.

2.2 JSON dan fromJson

JSON merupakan format pertukaran data yang umum digunakan oleh API. Respons JSON perlu diuraikan menjadi struktur data Dart. Konstruktor fromJson membantu mengubah Map hasil penguraian JSON menjadi objek model yang memiliki properti dengan tipe yang lebih jelas.

2.3 Future, async, dan await

Future merepresentasikan hasil yang akan tersedia kemudian. Fungsi async dapat menggunakan await untuk menunggu hasil operasi asinkron tanpa menghentikan responsivitas antarmuka aplikasi.

2.4 FutureBuilder

FutureBuilder membangun tampilan berdasarkan keadaan Future. Melalui snapshot, aplikasi dapat memeriksa apakah data sedang dimuat, terjadi kesalahan, atau hasil sudah tersedia.

2.5 Ringkasan Konsep

Konsep	Fungsi
http.get(uri)	Mengirim permintaan HTTP GET.
jsonDecode(body)	Mengubah teks JSON menjadi struktur data Dart.
fromJson	Mengubah Map menjadi objek model.
FutureBuilder	Menampilkan UI berdasarkan status Future.
snapshot	Menyediakan status, data, dan informasi error.

3. Persiapan dan Konfigurasi

3.1 Alat dan Bahan

- Flutter SDK dan editor kode.
- Emulator Android atau perangkat fisik.
- Koneksi internet.
- API publik JSONPlaceholder.

3.2 Membuat Proyek

Buat proyek Flutter baru melalui terminal:

```
flutter create praktikum_4
```

Masuk ke folder proyek, lalu tambahkan paket HTTP:

```
cd praktikum_4
flutter pub add http
```

3.3 Izin Internet Android

Jika diperlukan untuk build Android release, tambahkan izin berikut di AndroidManifest.xml, tepat sebelum tag <application>:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

4. Implementasi Praktikum

Untuk setiap bagian, gunakan pola penulisan: tujuan, langkah implementasi, penjelasan teknis, hasil yang diharapkan, dan bukti screenshot. Tempelkan kode lengkap atau potongan kode yang relevan sesuai implementasi yang dikerjakan.

4.1 Bagian A : Mengenal Future dan FutureBuilder

Tujuan dan langkah:

Membuat fungsi ambilSalam() yang menunggu dua detik sebelum mengembalikan teks. FutureBuilder menampilkan indikator loading selama menunggu, lalu menampilkan hasil. Future disimpan pada initState() agar tidak dibuat ulang setiap build.

Hasil pengujian: saat membuka aplikasi untuk pertama kali, muncul buffering selama 2 detik lalu menampilkan pesan halo dari masa depan.



4.2 Bagian B : Menyiapkan Paket HTTP

Tujuan dan langkah: Menambahkan paket http dan memeriksa izin internet Android bila dibutuhkan. Pastikan dependensi berhasil dipasang sebelum melanjutkan.

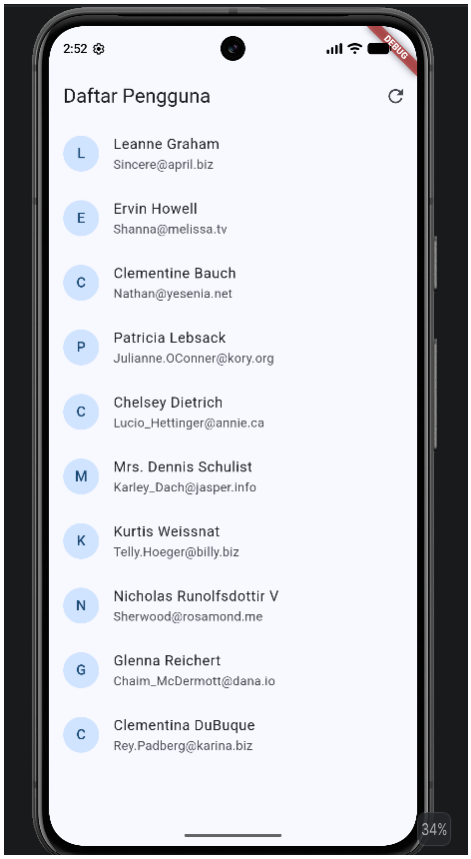
```
PS C:\Users\LENOVO LOQ\pemmob\pert4\praktikum4> flutter pub add http
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
+ http 1.6.0
+ http_parser 4.1.2
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.14 available)
+ typed_data 1.4.0
  vector_math 2.4.0 (2.4.3 available)
+ web 1.1.1
```

4.3 Bagian C : Model dan Fungsi Pengambil Data

Membuat class Pengguna dengan properti id, name, email, phone, dan website. Konstruktor Pengguna.fromJson() memetakan data JSON ke objek model. Fungsi ambilPengguna() mengirim GET ke endpoint /users dan memeriksa status respons sebelum menguraikan JSON.

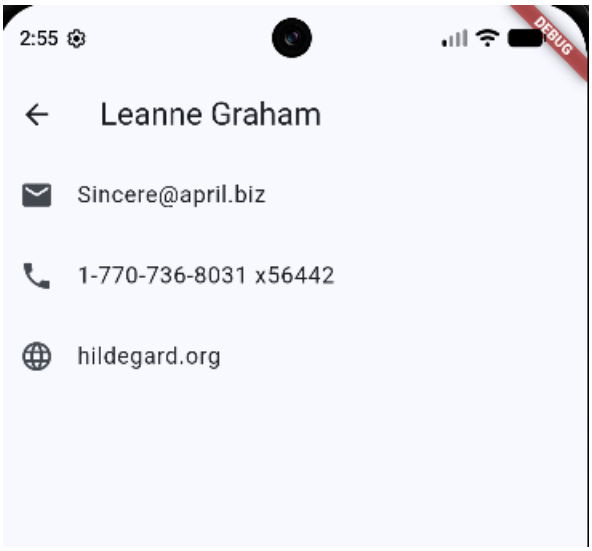
4.4 Bagian D : Menampilkan Data dengan FutureBuilder

Membuat PenggunaPage untuk menangani status loading, error, dan data. Daftar ditampilkan menggunakan ListView.builder. Tombol refresh memanggil fungsi pemuatan ulang, sedangkan tombol Coba lagi ditampilkan ketika terjadi kesalahan.



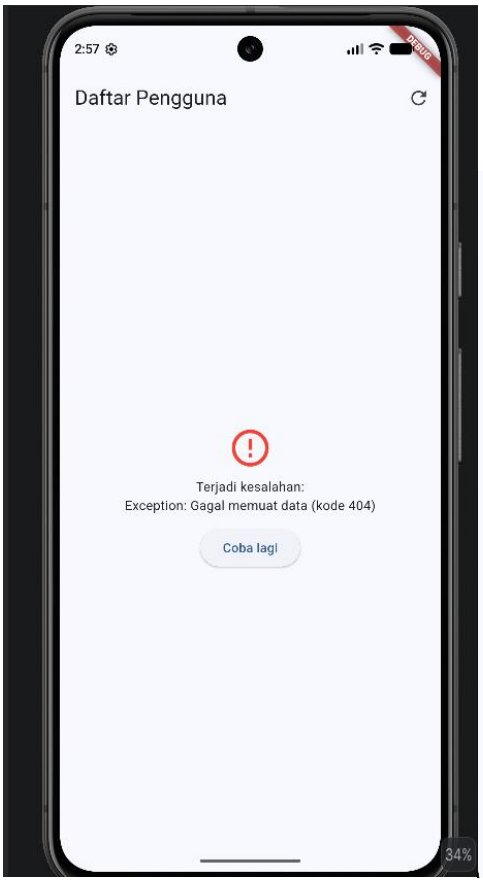
4.5 Bagian E : Halaman Detail

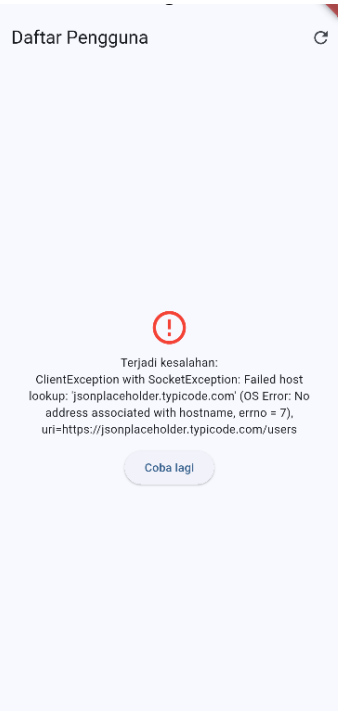
Ketika pengguna dipilih, aplikasi membuka DetailPenggunaPage dan menampilkan email, nomor telepon, serta website dari objek Pengguna yang dipilih.



4.6 Bagian F : Percobaan Penanganan Galat

Ubah endpoint menjadi /userz untuk mengamati respons 404, lalu kembalikan endpoint yang benar. Berikutnya, matikan koneksi internet dan tekan refresh untuk mengamati galat jaringan. Catat hasil dan langkah pemulihan.





Gambar sebelah kiri adalah tampilan ketika url user menjadi userz, dan gambar sebelah kanan adalah tampilan ketika koneksi internet emulator hp dimatikan.

5. Hasil Pengujian

Isi tabel berikut berdasarkan hasil eksekusi aplikasi. Jangan menyatakan pengujian berhasil sebelum hasilnya benar-benar diamati.

No.	Skenario	Hasil yang Diharapkan	Hasil Aktual
1	Aplikasi pertama kali dibuka	Indikator loading muncul, kemudian teks salam.	Aplikasi berhasil muncul setelah buffering 2 detik
2	Mengambil endpoint /users	Daftar pengguna ditampilkan.	Aplikasi menampilkan daftar pengguna dari URL
3	Memilih salah satu pengguna	Halaman detail tampil.	Email, no telp, dan website muncul di detail pengguna
4	Endpoint salah (/userz)	Pesan galat ditampilkan dan tersedia opsi mencoba lagi.	Muncul notifikasi error 404
5	Internet dimatikan	Galat jaringan ditampilkan.	Muncul notifikasi error
6	Refresh setelah koneksi pulih	Data dapat dimuat kembali.	Aplikasi kembali normal

6. Latihan Mandiri

Modul meminta pengembangan berikut. Dokumentasikan kode, hasil, dan screenshot setiap latihan.

- Menambahkan username dan kota (json['address']['city']) pada model dan halaman detail.

: Disini kita menambah atribut pada model Pengguna yaitu username dan city.

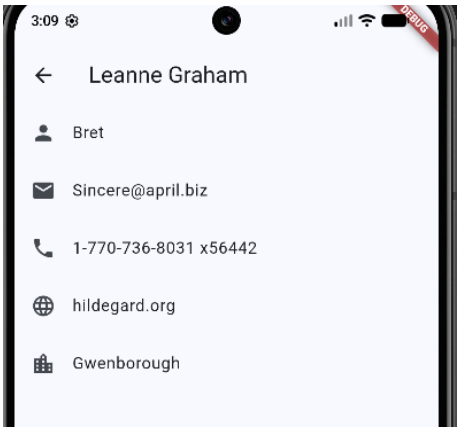
```
class Pengguna {
  final int id;
  final String name;
  final String username;
  final String email;
  final String phone;
  final String website;
  final String city;
}
```

Lalu menyesuaikan constructor dan mengambil data dari JSON menggunakan fromJson

```
factory Pengguna.fromJson(Map<String, dynamic> json) {
  return Pengguna(
    id: json['id'] as int,
    name: json['name'] as String,
    username: json['username'] as String,
    email: json['email'] as String,
    phone: json['phone'] as String,
    website: json['website'] as String,
```

```
            city: json['address']['city'] as String,
          );
        }
```

Hasil :



- Membungkus ListView dengan RefreshIndicator untuk memuat ulang melalui tarik ke bawah.
  : Widget ListView dibungkus menggunakan RefreshIndicator agar pengguna dapat memperbarui data dengan cara menarik daftar ke bawah (pull-to-refresh). Ketika gerakan tersebut dilakukan, fungsi pemuatan ulang akan dijalankan untuk mengambil data terbaru dari REST API. Selama proses berlangsung, aplikasi menampilkan indikator loading hingga data berhasil dimuat kembali.

```
return RefreshIndicator(
            onRefresh: () async {
              _muatUlang();
              await _future;
            },
            child: ListView.builder(
              itemCount: data.length,
              itemBuilder: (context, i) {
             final p = data[i];
```

- Menampilkan teks “Tidak ada data” jika daftar kosong.
  Pada bagian ini, aplikasi memeriksa apakah daftar pengguna yang berhasil diambil dari REST API kosong menggunakan kondisi data.isEmpty. Jika tidak ada data yang tersedia, aplikasi akan menampilkan teks “Tidak ada data” di bagian tengah layar.

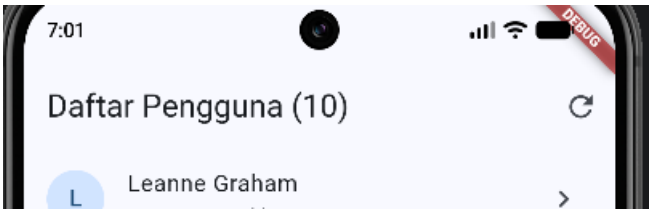
```
             if (data.isEmpty) {
                        return const Center(
                          child: Text('Tidak ada data'),
                        );
                    }
```

- Menampilkan jumlah pengguna pada AppBar setelah data berhasil dimuat.
  : Disini di bagian atas aplikasi, muncul jumlah daftar pengguna sesuai yang tertera di aplikasi.

```
void _muatUlang() {
    setState(() {
      _future = ambilPengguna();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Pengguna'),
        actions: [
          IconButton(
            icon: const Icon(Icons.refresh),
            onPressed: _muatUlang,
          ),
        ],
       ),
```

Hasil:



7. Tugas Praktikum: Aplikasi Daftar Postingan

7.1 Deskripsi Tugas

Buat aplikasi Daftar Postingan menggunakan endpoint /posts untuk daftar postingan dan /posts/{id}/comments untuk komentar dari postingan yang dipilih.

7.2 Ketentuan:

- Halaman utama menampilkan daftar postingan, judul, dan potongan isi menggunakan FutureBuilder:

```
class _PostPageState extends State<PostPage> {
  late Future<List<Post>> _future;

  @override
  void initState() {
    super.initState();
    _future = ambilPost();
}
```

- Ketika postingan dipilih, halaman detail menampilkan isi lengkap dan daftar komentar dari Future kedua:

```
onTap: () {
  Navigator.push(
```

```
      context,
      MaterialPageRoute(
        builder: (_) => DetailPostPage(
          post: post,
        ),
      ),
    );
  },
```

- Membuat model Post dan Komentar menggunakan fromJson.
  : Model Post menampung ID pengguna, ID postingan, judul, dan isi postingan. Fungsi Post.fromJson() mengubah data JSON dari API menjadi objek Post. Model Komentar menampung ID postingan, ID komentar, nama pengirim, email, dan isi komentar. Fungsi Komentar.fromJson() digunakan untuk mengubah data JSON komentar menjadi objek Komentar.
- Memisahkan kode pengambilan data dari kode UI.
  Ada dua fungsi utama untuk mengambil data dari REST API, yaitu mengambil daftar postingan dari API dan mengambil data komentar dari endpoint.

```
Future<List<Post>> ambilPost() async {
  final uri = Uri.parse(
    'https://jsonplaceholder.typicode.com/posts',
  );

  final response = await http
      .get(uri)
      .timeout(const Duration(seconds: 10));

  if (response.statusCode != 200) {
    throw Exception(
      'Gagal memuat postingan (kode ${response.statusCode})',
    );
  }

  final List<dynamic> data = jsonDecode(response.body);

  return data
      .map(
        (e) => Post.fromJson(
          e as Map<String, dynamic>,
        ),
      )
      .toList();
}
```

Kode diatas adalah Mengambil daftar postingan dari endpoint /posts.

```
Future<List<Post>> ambilPost() async {
  final uri = Uri.parse(
    'https://jsonplaceholder.typicode.com/posts',
  );
```

```
    final response = await http
        .get(uri)
        .timeout(const Duration(seconds: 10));

    if (response.statusCode != 200) {
      throw Exception(
        'Gagal memuat postingan (kode ${response.statusCode})',
      );
    }

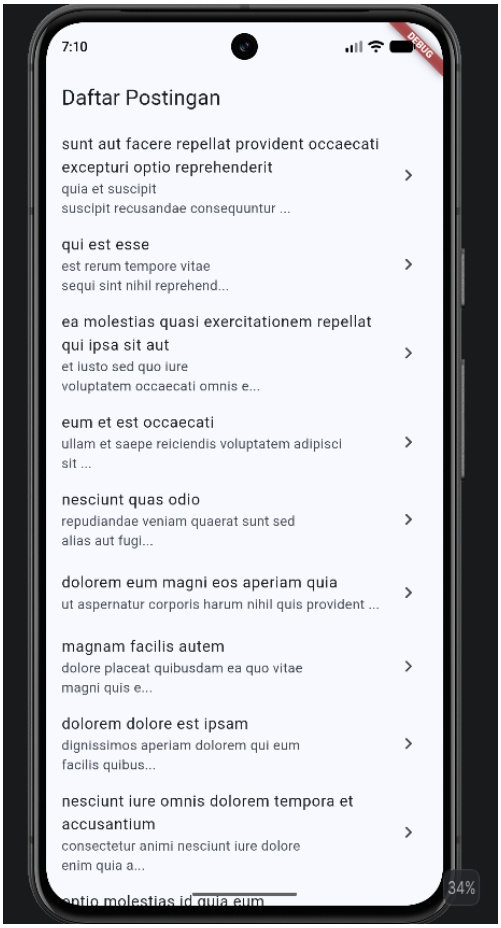
    final List<dynamic> data = jsonDecode(response.body);

    return data
        .map(
          (e) => Post.fromJson(
            e as Map<String, dynamic>,
          ),
        )
        .toList();
  }
```

Sementara code diatas adalah mengambil komentar dari endpoint /posts/{id}/comments.
Kedua fungsi tersebut memisahkan proses permintaan HTTP, pemeriksaan respons, penguraian JSON, dan konversi model dari kode tampilan. Perbedaannya, ambilKomentar() menerima parameter postId agar komentar yang diambil sesuai dengan postingan yang dipilih.

- Menangani loading, error, dan tombol coba lagi pada kedua halaman.
  : Kedua tombol menggunakan fungsi pemuatan ulang masing-masing. Halaman utama mengambil ulang daftar postingan, sedangkan halaman detail mengambil ulang komentar dari postingan yang sedang dibuka.

7.3 Hasil



8. Pembahasan dan Refleksi

1. Apa yang terjadi bila Future dibuat di dalam build() dan bukan di initState()? Mengapa?

Jawaban: Jika Future dibuat di dalam build() dan bukan di initState(), maka Future akan dibuat ulang setiap kali build() dijalankan kembali, sehingga pengambilan data dapat terpanggil berulang-ulang, UI dapat kembali ke status loading, data lama dapat ter-reset.

2. Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode? Mengapa keduanya diperlukan?

Jawaban: snapshot.hasError menunjukkan bahwa Future mengalami error atau melempar exception, misalnya karena koneksi gagal, timeout, atau parsing JSON gagal, sedangkan response.statusCode adalah kode status HTTP yang hanya tersedia jika respons server benar-benar diterima, seperti 200 untuk sukses, 404 untuk tidak ditemukan, dan 500 untuk kesalahan server.

3. Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai Map?

Jawaban: JSON sebaiknya diubah ke class model karena Map terlalu bebas dan rawan salah ketik key atau salah tipe data. Dengan class model dan fromJson, data jadi lebih jelas, lebih aman, dan lebih mudah dipakai di UI. Kode juga lebih rapi dan mudah diperbaiki kalau ada perubahan.

4. Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?

Jawaban: UI perlu punya status loading dan error karena mengambil data dari internet butuh waktu dan bisa gagal. Kalau hanya ada status data, pengguna bisa melihat layar kosong dan mengira aplikasi hang. Dengan loading, pengguna tahu aplikasi sedang memuat. Dengan error dan tombol coba lagi, pengguna tahu ada masalah dan bisa mencoba ulang.

9. Kesimpulan

Modul ini membahas cara mengambil data dari REST API di Flutter menggunakan paket http, pemrograman asinkron (Future, async, await), dan FutureBuilder. Mahasiswa dilatih mengubah JSON menjadi class model dengan fromJson, lalu menampilkannya di UI dengan tiga keadaan utama: loading, error, dan data.

10. Referensi dan Lampiran

Referensi dari modul:

- Flutter : Fetch data from the internet: https://docs.flutter.dev/cookbook/networking/fetch-data
- Dart : Asynchronous programming / async-await: https://dart.dev/codelabs/async-await
- Paket http: https://pub.dev/packages/http
- JSONPlaceholder: https://jsonplaceholder.typicode.com